DOCUMENTO DE ARQUITECTURA E INGENIERÍA DE SOFTWARE

Plataforma de Movilidad y Ecoturismo de Villagarzón

Especificación Integral de Arquitectura de Microservicios, Metodología de Desarrollo, Stack Tecnológico y Requerimientos del Sistema

Este documento constituye el plan técnico y arquitectónico oficial para el diseño, construcción y despliegue de la plataforma integral de transporte bajo demanda y fomento ecoturístico en el municipio de Villagarzón, departamento del Putumayo. Tomando como referencia los patrones operativos de soluciones globales de movilidad urbana, la plataforma introduce adaptaciones de ingeniería específicas para la geografía amazónica, la conectividad intermitente y la articulación entre mototaxis urbanos, flotas de transporte terrestre, terminal aeroportuaria y circuitos de ecoturismo regional.

Proyecto	Plataforma Regional de Movilidad y Turismo (Villagarzón - Putumayo)
Versión del Documento	1.0.0 — Arquitectura Base y Requerimientos de Construcción
Fecha de Emisión	Noviembre de 2026
Patrón Arquitectónico	Microservicios Contenerizados con Docker / Event-Driven Messaging
Stack Tecnológico Principal	Node.js & TypeScript (Backend) · React Native & React (Apps & Web) · PostgreSQL/PostGIS & Redis
Ámbito Geográfico	Villagarzón (Casco Urbano, Aeropuerto Cananguchal VGZ, Veredas Turísticas y Corredor Mocoa)
Estado de Ingeniería	Aprobado para Inicio de Desarrollo e Implementación

Documento confidencial para el equipo de desarrollo, gestión de producto y partes interesadas operativas de la plataforma. Prohibida su reproducción o distribución no autorizada.

1. Contexto Estratégico, Visión y Alcance Operativo

El municipio de Villagarzón ocupa una posición neurálgica en el departamento del Putumayo, funcionando como el principal polo de interconexión aérea del departamento a través del **Aeropuerto Cananguchal (VGZ)** y como punto de enlace vial entre Mocoa, Puerto Asís, Orito y el Valle del Guamuez. Asimismo, su riqueza hídrica y paisajística alberga atractivos de alto valor ecoturístico como el Río Guineo, Pozo Azul, las Cascadas de Vica y Mohano, y reservas naturales comunitarias.

A pesar de esta vitalidad económica y turística, la movilidad local enfrenta brechas operativas significativas: dispersión de prestadores de transporte informal (mototaxis y taxis no centralizados), falta de certidumbre tarifaria para el usuario local y el turista recién arribado al aeropuerto, nula visibilidad de disponibilidad de vehículos en tiempo real y ausencia de herramientas digitales para programar traslados seguros hacia zonas naturales.

La plataforma resuelve estas problemáticas mediante un ecosistema digital multi-canal (Aplicación Móvil para Pasajeros, Aplicación Móvil para Conductores, Portal Turístico Web y Panel de Control Administrativo). La solución equilibra las exigencias operativas de un servicio de movilidad inmediata en el casco urbano con la flexibilidad requerida para contratar rutas turísticas programadas y traslados intermunicipales.

1.1. Modalidades de Transporte Integradas

Modalidad	Descripción y Radio Operativo	Esquema de Cobro
Mototaxi Urbano	Transporte rápido individual dentro del perímetro urbano de Villagarzón y barrios periféricos.	Tarifa base plana diurna/nocturna con recargo paramétrico por zonas periféricas.
Taxi Convencional	Vehículos tipo sedán/hatchback para transporte urbano, traslados con equipaje y conexión al Aeropuerto Cananguchal.	Tarifa calculada por tiempo/distancia o precio pactado sugerido con recargo aeroportuario.
Campero / 4×4 Rural	Camionetas y vehículos todo terreno adecuados para vías terciarias destapadas y balnearios rurales.	Tarifa fija por destino rural o contratación por horas con soporte para grupos de hasta 8 pasajeros.
Circuito Ecoturístico	Servicio combinado de traslado ida/vuelta con espera garantizada y opción de guía local certificado.	Paquete turístico prepagado o reservado con anticipación mediante el portal web o app.

2. Metodología de Desarrollo y Gobernanza de Ingeniería

Para asegurar una entrega incremental, controlada y de alta calidad técnica, el desarrollo del proyecto se ejecutará bajo el marco ágil **Scrum adaptado** en ciclos de iteración rápida (sprints de 2 semanas), complementado con prácticas de **Integración Continua y Despliegue Continuo (CI/CD)** y una arquitectura de repositorio unificada.

2.1. Gestión de Repositorio y Monorepo con Turborepo

Se implementa una estructura de *Monorepo* para unificar la base de código del backend y de los clientes frontend. Esto garantiza que las definiciones de tipos TypeScript, los modelos de datos y las funciones utilitarias de geolocalización sean compartidas sin discrepancias entre clientes y servidores:

- `/apps/mobile-passenger:` Aplicación móvil de pasajeros construida en React Native.
- `/apps/mobile-driver:` Aplicación móvil de conductores (mototaxis y taxis) en React Native con servicios de geolocalización en segundo plano.
- `/apps/web-admin:` Panel web administrativo y consola de despacho en tiempo real en React / Next.js.
- `/apps/web-tourism:` Portal web público de consulta, promoción de destinos y reserva de paquetes ecoturísticos.
- `/services/*:` Servicios de backend independientes (Gateway, Auth, Trips, Tracking, Pricing, etc.).
- `/packages/shared-types:` Interfaces TypeScript unificadas (DTOs, contratos de API, payloads de WebSocket).
- `/packages/geo-utils:` Funciones matemáticas de cálculo de distancias Haversine, decodificación de polilíneas y validación de geocercas en Putumayo.

2.2. Flujo de Trabajo Gitflow y Control de Calidad

El versionamiento del código se regirá por Gitflow estructurado:

- **Rama main:** Código en producción, estrictamente estable, respaldado por versiones etiquetadas (tags SemVer).
- **Rama develop:** Rama de integración para el entorno de Staging y pruebas de aceptación de usuario (UAT).
- **Ramas feature/*:** Ramas de desarrollo de requerimientos específicos originadas y fusionadas hacia develop mediante Pull Requests (PR).
- **Ramas hotfix/*:** Correcciones críticas emergentes que se aplican directamente a main y se propagan a develop.

Cada Pull Request exige la aprobación de al menos dos ingenieros revisores, cumplimiento de cobertura de pruebas unitarias superior al 80% (medido con Jest) y validación estricta de análisis estático (ESLint y Prettier).

---

3. Arquitectura del Sistema: Microservicios en Contenedores Docker

La plataforma adopta una **Arquitectura de Microservicios orientada a eventos (EDA - Event-Driven Architecture)**, en la que cada dominio funcional del negocio opera como un servicio autónomo empaquetado en un contenedor Docker con límites claros de responsabilidad y persistencia dedicada.

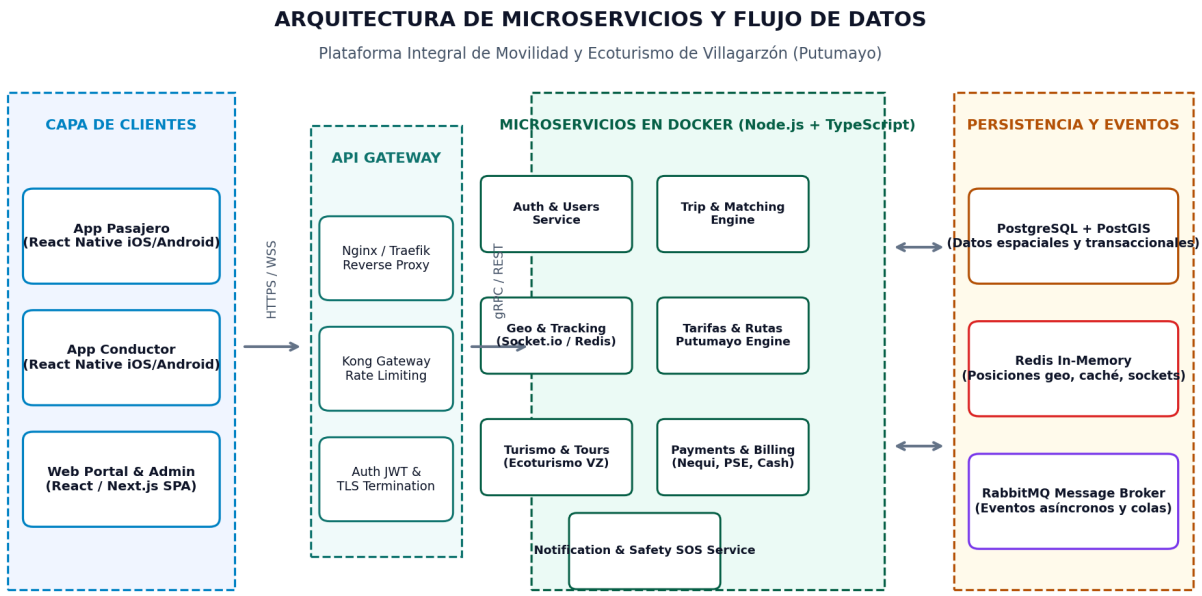


Figura 1: Topología de Microservicios, Enrutamiento API Gateway y Capas de Persistencia en Docker.

3.1. Catálogo y Responsabilidad de Microservicios Core

Microservicio	Responsabilidad Funcional y Dominio	Tecnologías y Persistencia
API Gateway & Reverse Proxy	Punto único de entrada, enrutamiento inverso, terminación SSL/TLS, validación de JWT, balanceo de carga y limitación de tasa (rate limiting contra ataques DDoS).	Nginx / Kong Gateway, Redis (Rate Limit counters)
Auth & Identity Service	Registro y autenticación de usuarios, emisión de tokens JWT (Access & Refresh), validación de números telefónicos mediante OTP por SMS/WhatsApp, gestión de roles RBAC.	Node.js / TypeScript, Fastify, PostgreSQL (Schema Auth), Argon2id
Driver & Compliance Service	Onboarding de conductores, validación de licencias de conducción, certificados SOAT, revisión tecnomecánica, antecedentes judiciales y estado de habilitación de flota.	Node.js / TypeScript, PostgreSQL, MinIO/S3 (almacenamiento de fotos de documentos)
Geolocation & Tracking Service	Recepción continua de coordenadas GPS de conductores activos (frecuencia 1.5s), indexación espacial en memoria y retransmisión de posición a pasajeros asignados.	Node.js / TypeScript, Socket.io Engine, Redis Geospatial (GEOADD / GEORADIUS)
Trip Matching Engine	Orquestación del ciclo de vida del viaje: recepción de solicitud, búsqueda de conductores más cercanos por georradio, emisión de ofertas con timeout (15s) y asignación.	Node.js / TypeScript, PostgreSQL + PostGIS (historial), Redis, RabbitMQ

Microservicio	Responsabilidad Funcional y Dominio	Tecnologías y Persistencia
Tarifas & Putumayo Pricing Engine	Cálculo dinámico y paramétrico de tarifas: costo base urbano, costo por kilómetro y minuto, recargo nocturno, recargo al Aeropuerto Cananguchal y multiplicador por vía destapada.	Node.js / TypeScript, PostgreSQL (Tablas de tarifas paramétricas y geocercas)
Turismo & Circuitos Service	Catálogo de destinos naturales de Villagarzón, gestión de cupos, reserva anticipada de camionetas camperas, contratación de guías locales y cobro de paquetes ecoturísticos.	Node.js / TypeScript, PostgreSQL (Schema Turismo), Redis (bloqueo de cupos)
Payments & Wallet Service	Procesamiento de pagos en efectivo, gestión de billeteras de conductores (billetera prepago para descuento de comisión del 10%), pasarelas Nequi, Daviplata y PSE.	Node.js / TypeScript, PostgreSQL (transacciones ACID contables), Wompi / PayU SDK
Safety & Emergency SOS Service	Gestión del botón de pánico en tiempo real, enlace de seguimiento cifrado compartible por WhatsApp, alertas automáticas a la central y enlace con la Policía de Villagarzón.	Node.js / TypeScript, Socket.io, Twilio SMS API / Webhook policial
Notification Service	Envío de notificaciones push transaccionales a conductores y pasajeros (cambio de estado del viaje, asignación, llegada del vehículo) y fallback de SMS para zonas rurales.	Node.js / TypeScript, Firebase Cloud Messaging (FCM), AWS SNS / Twilio

3.2. Estrategia de Mensajería Asíncrona y Event Bus

Para desacoplar las operaciones críticas en el tiempo de aquellas complementarias, se utiliza un bus de mensajes basado en **RabbitMQ**. Cuando ocurre un evento de dominio clave (por ejemplo, `TripCompletedEvent`), el *Trip Matching Engine* emite el evento al bus; de forma asíncrona y paralela, el *Payments Service* liquida la comisión, el *Notification Service* envía el recibo digital al pasajero, y el servicio de analítica actualiza los indicadores del conductor, garantizando que ninguna falla secundaria bloquee la confirmación del viaje.

---

4. Stack Tecnológico y Lenguajes de Programación

La selección tecnológica responde a criterios estrictos de alto rendimiento para procesamiento asíncrono en tiempo real, interoperabilidad entre plataformas móviles y web, y reducción de la sobrecarga operativa mediante tipado estricto extremo a extremo.

4.1. Justificación del Ecosistema de Desarrollo

- **Backend (Node.js v20+ LTS con TypeScript):** El bucle de eventos no bloqueante de Node.js es el estándar de la industria para arquitecturas intensivas en operaciones de entrada/salida (I/O) en tiempo real, como el rastreo continuo de flotas y el intercambio bidireccional por WebSockets. El uso de TypeScript en modo estricto previene errores en tiempo de compilación y asegura contratos formales entre microservicios.

- **Frontend Móvil (React Native con TypeScript):** Permite compilar aplicaciones nativas de alto rendimiento para Android e iOS compartiendo más del 80% del código fuente entre las apps de pasajero y conductor. Su compatibilidad con hilos de geolocalización en background nativos garantiza una precisión de rastreo GPS continua sin agotar la batería del dispositivo del conductor.
- **Frontend Web (React 18+ y Next.js):** Proporciona una interfaz reactiva, modular y ligera para el panel de administración central, la consola de despacho de despachadores locales y el portal de turismo abierto al público global.
- **Capa de Datos Geoespacial (PostgreSQL 16 + PostGIS):** PostGIS es el motor de base de datos relacional y geoespacial por excelencia, permitiendo ejecutar consultas indexadas de proximidad euclidiana y geodésica (`ST_DWithin`, `ST_Distance`) y validación de geometrías de geocercas en milisegundos.
- **Caché en Memoria y Geo-Indexación Volátil (Redis 7):** El seguimiento de miles de posiciones en tiempo real no debe sobrecargar el disco de la base de datos relacional. Redis almacena los puntos GPS en estructuras nativas de conjuntos ordenados geoespaciales (`GEOADD` y `GEORADIUS`), ofreciendo lecturas sub-milisegundo.

4.2. Matriz de Componentes del Stack

Capa	Tecnología Seleccionada	Función Principal
Lenguaje Base	TypeScript 5.x sobre Node.js 20 LTS	Tipado estático transversal en microservicios, móviles y web.
Framework HTTP	Fastify / Express	Microservicios REST ligeros de baja sobrecarga y alta tasa de req/s.
Sockets & Tiempo Real	Socket.io v4 con Redis Adapter	Conexiones bidireccionales en vivo entre apps y servidores.
Base de Datos Relacional	PostgreSQL 16 con extensión PostGIS	Persistencia transaccional ACID, usuarios, viajes y datos espaciales.
Caché y Geo-Índices	Redis 7 In-Memory Cluster	Caché de sesiones, token blacklist, colas de despacho y geo-tracking.
Message Broker	RabbitMQ 3.13	Encolamiento y distribución asíncrona de eventos de dominio.
Móvil (Driver / Passenger)	React Native 0.74+, React Navigation, Mapbox GL	Aplicaciones móviles de alta respuesta táctil con mapas offline parciales.
Web (Admin / Portal)	Next.js 14, Tailwind CSS, TanStack Query	Portal público SEO-friendly de ecoturismo y panel administrativo.
Contenedores y Red	Docker Engine, Docker Compose, Nginx Reverse Proxy	Empaquetamiento hermético, aislamiento de dependencias y balanceo.

5. Requerimientos Funcionales del Sistema (RF)

Los requerimientos funcionales definen de manera precisa los comportamientos, reglas de negocio y flujos que el sistema ejecutará para los diferentes actores (Pasajeros, Conductores, Operadores Turísticos y Administradores).

5.1. Matriz Detallada de Requerimientos Funcionales

Código	Módulo	Descripción y Criterio de Aceptación	Prioridad
RF-01	Autenticación	Registro e inicio de sesión de pasajeros y conductores mediante número de teléfono celular con validación obligatoria de código OTP vía SMS o WhatsApp.	Must Have (Obligatorio)
RF-02	Onboarding Conductor	Carga fotográfica de documentos: cédula de ciudadanía, licencia de conducción vigente, tarjeta de propiedad, SOAT y revisión tecnomecánica. El conductor solo podrá activarse tras verificación del administrador.	Must Have (Obligatorio)
RF-03	Rastreo en Tiempo Real	La app del conductor transmite sus coordenadas de geolocalización cada 1.5 segundos cuando se encuentra en estado 'Disponible' o 'En Viaje', reflejándose en el mapa del pasajero con interpolación suave.	Must Have (Obligatorio)
RF-04	Despacho y Asignación	Algoritmo de búsqueda que localiza conductores disponibles en un radio inicial de 2 km en Villagarzón. Envía la oferta con cuenta regresiva de 15 segundos. Si se rechaza o expira, escala al siguiente conductor más cercano.	Must Have (Obligatorio)
RF-05	Motor de Tarifas	Cálculo automático del costo estimado del viaje antes de confirmar: Tarifa Base Urbana ($3.000 COP mototaxi / $6.000 COP taxi) + recargo nocturno (después de las 8:00 PM) + recargo al Aeropuerto Cananguchal ($5.000 COP adicionales).	Must Have (Obligatorio)
RF-06	Estados del Viaje	Máquina de estados finita: Solicitado → Aceptado → Conductor en Camino → Conductor en Sitio → En Curso → Finalizado → Calificado. Registro de timestamps y geolocalizaciones en cada transición.	Must Have (Obligatorio)
RF-07	Ecoturismo y Destinos	Módulo específico con catálogo interactivo de sitios naturales de Villagarzón (Pozo Azul, Río Guineo, Vica, Mohano). Permite cotizar y reservar vehículos 4×4 o camperos por viaje de ida, ida/vuelta o jornada completa.	Must Have (Obligatorio)
RF-08	Reserva Programada	Los turistas pueden programar servicios de transporte con hasta 7 días de antelación para llegadas de vuelos en el Aeropuerto Cananguchal o excursiones a primera hora.	Should Have (Deseable)

Código	Módulo	Descripción y Criterio de Aceptación	Prioridad
RF-09	Pagos Multi-Método	Soporte nativo de pago en efectivo con cálculo del cambio requerido, integración directa con billeteras Nequi y Daviplata (generación de QR dinámico), y pasarela PSE/Tarjetas.	Must Have (Obligatorio)
RF-10	Billetera de Conductor	Sistema de saldo prepagado en la app del conductor. Cada viaje finalizado descuenta automáticamente la comisión de la plataforma (ej. 10%). Si el saldo es inferior a $5.000 COP, el conductor pasa a estado inactivo para recibir viajes en efectivo.	Must Have (Obligatorio)
RF-11	Botón SOS y Pánico	Acceso inmediato a un botón de emergencia durante el viaje. Al activarse, envía la ubicación en vivo por SMS a 3 contactos de confianza y emite una alerta crítica prioritaria en el panel de control administrativo.	Must Have (Obligatorio)
RF-12	Calificación y Reseñas	Evaluación mutua y bidireccional (1 a 5 estrellas) entre conductor y pasajero al culminar el trayecto, con etiquetas de retroalimentación obligatorias en caso de puntajes menores a 3 estrellas.	Must Have (Obligatorio)
RF-13	Panel Administrativo	Dashboard web para visualizar la flota en tiempo real sobre el mapa de Villagarzón, aprobar o suspender conductores, modificar tablas tarifarias y exportar reportes operativos.	Must Have (Obligatorio)

6. Requerimientos No Funcionales (RNF) y Estándares de Calidad

Los requerimientos no funcionales garantizan que el sistema no solo cumpla sus funciones de negocio, sino que lo haga bajo estrictos criterios de seguridad, velocidad, confiabilidad y resiliencia ante los retos específicos de infraestructura del Putumayo.

6.1. Especificación de Requerimientos No Funcionales

Categoría	Métrica / Requerimiento No Funcional	Mecanismo de Cumplimiento
Rendimiento	Latencia de transmisión de posición GPS < 200 ms. Tiempo de respuesta de endpoints de API p95 < 150 ms bajo una carga de 1.000 usuarios concurrentes.	Protocolo WebSockets nativo, indexación Redis en memoria y queries PostgreSQL optimizadas con índices espaciales GIST.
Disponibilidad	Disponibilidad del servicio del 99.9% (equivalente a menos de 43 minutos de indisponibilidad imprevista al mes).	Orquestación en contenedores Docker con políticas de reinicio automático, balanceadores Nginx redundantes y healthchecks periódicos.

Categoría	Métrica / Requerimiento No Funcional	Mecanismo de Cumplimiento
Seguridad de Datos	Cifrado completo de datos en tránsito mediante TLS 1.3 y en reposo mediante AES-256. Autenticación con tokens JWT firmados asimétricamente (RS256) con expiración de 15 minutos.	Certificados SSL Let's Encrypt / Cloudflare, hashing de credenciales con Argon2id y cumplimiento de la Ley 1581 de 2012 de Habeas Data Colombia.
Resiliencia Amazónica (Offline-First)	En zonas rurales de Villagarzón con pérdida temporal de conectividad celular (2G o sin señal), la app del conductor debe registrar y encolar las coordenadas de la ruta localmente.	Base de datos local SQLite / WatermelonDB en la app móvil con sincronización transaccional automática tan pronto se restablezca la conexión de red 3G/4G.
Escalabilidad	Capacidad de escalar horizontalmente de 1 a 10 réplicas de microservicios sin pérdida de estado durante picos de demanda (festividades locales, temporadas de vacaciones).	Arquitectura 100% Stateless en los contenedores de aplicación; delegación de estado a Redis y PostgreSQL con pool de conexiones PgBouncer.
Consumo de Batería	La app móvil del conductor no debe consumir más del 12% de batería por hora de rastreo continuo en segundo plano.	Ajuste dinámico del intervalo de GPS (geofencing inteligente: mayor frecuencia en movimiento, menor frecuencia en detención).

7. Plan de Despliegue, Pipeline CI/CD y Hoja de Ruta de Escalabilidad

El despliegue de la solución está estructurado para mitigar riesgos técnicos y comerciales, iniciando con una validación intensiva en el núcleo urbano de Villagarzón y expandiéndose hacia el ecosistema turístico y corredores intermunicipales.

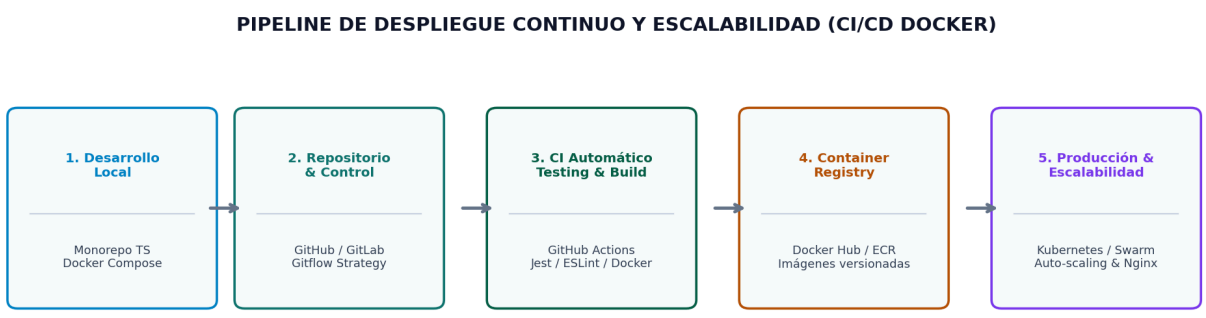


Figura 2: Ciclo de Integración y Despliegue Continuo Automatizado en Contenedores Docker.

7.1. Etapas de la Hoja de Ruta de Implementación

Fase	Objetivos Principales	Entregables Técnicos	Plazo Estimado
Fase 1: MVP Urbano y Aeropuerto	Lanzamiento del servicio base de mototaxis y taxis urbanos en Villagarzón y ruta fija hacia el Aeropuerto Cananguchal. Validación del matching en vivo y pagos en efectivo.	Apps Android (APK/Play Store), microservicios core en Docker, panel de control básico para la central de despacho local.	Meses 1 a 3
Fase 2: Ecoturismo y Pagos Móviles	Lanzamiento del portal web de turismo, integración de camionetas camperas para zonas de difícil acceso (Río Guineo, Cascadas), habilitación de pagos Nequi/Daviplata y billetera prepago.	Portal web Next.js, pasarelas de pago colombianas, soporte iOS en App Store, reservas anticipadas de tours con guías.	Meses 4 a 5
Fase 3: Expansión Regional Putumayo	Apertura de rutas intermunicipales (Villagarzón - Mocoa - Puerto Asís), algoritmos predictivos de demanda para vuelos comerciales y convenios con gremios transportadores.	Clúster de alta disponibilidad en Kubernetes/Swarm, analítica avanzada de rutas y panel especializado para empresas de transporte aliado.	Meses 6 en adelante

7.2. Topología de Infraestructura y Ambientes de Despliegue

Se configuran tres ambientes técnicos completamente segregados:

- **Ambiente de Desarrollo (Local / Dev):** Ejecutado directamente en las máquinas de los ingenieros mediante `docker-compose.dev.yml`, con hot-reload para Node.js y React Native con Expo Metro Bundler. Emulación de base de datos PostgreSQL local y Redis.
- **Ambiente de Staging / QA:** Servidor VPS dedicado (4 vCPU, 16 GB RAM) donde se despliega automáticamente cada cambio aprobado en la rama `develop`. Utilizado para pruebas de estrés, pruebas de campo GPS con mototaxistas en las calles de Villagarzón y homologación de pagos.
- **Ambiente de Producción:** Clúster contenerizado en la nube (AWS / DigitalOcean / Hetzner) con balanceo Nginx, certificados SSL con renovación automática, réplica de lectura en PostgreSQL, clúster Redis redundante y respaldos automáticos diarios de base de datos en almacenamiento S3 cifrado.

7.3. Observabilidad, Monitoreo y Métricas en Tiempo Real

Para garantizar una operación ininterrumpida y detectar anomalías antes de que afecten a los usuarios, se incorpora la siguiente suite de observabilidad:

- **Prometheus & Grafana:** Recolección continua de métricas de infraestructura (uso de CPU, consumo de memoria de contenedores Docker, conexiones activas de WebSockets por minuto y tiempo de latencia de endpoints).
- **Loki / Promtail:** Agregación centralizada de registros (logs) de todos los microservicios con correlación por ID de viaje (`tripId`), facilitando el rastreo de cualquier incidencia operativa en segundos.

- **Sentry:** Monitoreo en tiempo real de excepciones no controladas tanto en los microservicios backend como en las aplicaciones móviles de los usuarios en producción.

---

8. Matriz de Riesgos Técnicos, Mitigación y Cumplimiento Normativo

La operación de una plataforma tecnológica de movilidad en el contexto de Villagarzón y la región del Putumayo enfrenta desafíos operativos, geográficos y regulatorios que requieren medidas de mitigación de ingeniería claramente definidas:

Riesgo Identificado	Impacto	Estrategia de Mitigación de Ingeniería
Pérdida de conectividad celular en rutas ecoturísticas	Alto	Implementación estricta de arquitectura *Offline-First* con persistencia SQLite en el dispositivo móvil del conductor. Los viajes hacia atractivos como Pozo Azul o Río Guineo se inician con precarga de tarifas fijas y se sincronizan asíncronamente al retomar red 3G/4G.
Baja tasa de bancarización digital en la población local	Medio-Alto	Soporte prioritario para pagos en efectivo con control de recaudo mediante billetera prepago de comisiones para el conductor. Alianzas de recarga local vía puntos Efecty / SuperGiros y generación simplificada de códigos QR dinámicos para Nequi y Daviplata.
Resistencia de gremios de transporte tradicional	Alto	Modelo inclusivo no disruptivo que formaliza e integra a los mototaxis y taxis agremiados como socios conductores. Panel web para asociaciones locales que permite visualizar estadísticas y despacho corporativo sin desplazar su modelo de sustento.
Regulación de transporte y Habeas Data (Colombia)	Alto	Alineación estricta con el marco normativo del Ministerio de Transporte de Colombia y la Superintendencia de Transporte para plataformas de intermediación tecnológica. Cumplimiento integral de la Ley 1581 de 2012 para la custodia segura de datos de geolocalización.

8.1. Criterios de Aceptación Arquitectónica para Salida a Producción

Previo a la publicación oficial en tiendas de aplicaciones (Google Play y Apple App Store) y la activación en el casco urbano de Villagarzón, el sistema deberá superar las siguientes pruebas de certificación de ingeniería:

- **Prueba de Carga y Concurrencia:** Soporte comprobado de al menos 1.500 solicitudes de viaje simultáneas y 500 conexiones de tracking por WebSocket con uso de CPU inferior al 65% en contenedores Docker.
- **Validación de Precisión GPS:** Margen de error en cálculo de distancia y geocercas inferior a 15 metros en el perímetro urbano de Villagarzón y el corredor del Aeropuerto Cananguchal.
- **Auditoría de Seguridad y Pentesting:** Cero vulnerabilidades críticas o altas detectadas en el API Gateway y microservicios según el estándar OWASP Top 10.
- **Resiliencia y Failover:** Recuperación automática de contenedores caídos en menos de 10 segundos sin pérdida de sesiones activas de usuarios.